

Part A: Image Negative

Write a C program that:

1. Reads a grayscale `.pgm` image using the provided `pgm.o` library, as taught in class.
2. Computes the image negative using a sequential implementation.
3. Computes the same image negative using an OpenMP implementation.
4. Saves the resulting negative image.
5. Reports the execution time and speedup.

The input and output filenames may be specified directly in the source code.

For each pixel p , calculate the negative as:

$$p_{\text{negative}} = \text{maxval} - p$$

Requirements

- Include both the sequential and OpenMP implementations in the same C program.
- Use the provided `pgm.o` library for reading and writing images, as taught in class.
- Surround the sequential processing block with calls to `omp_get_wtime()` to calculate T_{seq} .
- Surround the OpenMP processing block with calls to `omp_get_wtime()` to calculate T_{par} .
- Use `#pragma omp parallel for` to parallelize the image-processing loop.
- Measure only the processing blocks; do not include image reading or writing in the measured time.
- Ensure that the sequential and OpenMP implementations produce identical results.
- Report:

$$T_{seq}, \quad T_{par}, \quad \text{Speedup} = \frac{T_{seq}}{T_{par}}$$

- Properly release all dynamically allocated memory.